🏗️ 模块剖析：App.vue (应用总底盘)

维度	描述
模块路径	`frontend/src/App.vue`
核心职责	定义应用全局布局、掌管系统初始化生命周期、维护全局时钟与通信状态
类比对象	**系统总底盘 / 主板总线**

🧱 业务黑盒 (Product Blueprint)

1. 核心职责

作为 Vue 应用的根组件，它在系统上电（加载）后第一个运行。它不负责具体的传感器显示，而是负责：

- **布局搭建**：焊好 Header（标题/状态灯）和 Sidebar（菜单栏）。
- **初始化心跳**：开启系统的第一声"脉搏"（1s 时间刷新）。
- **通信建立**：通知数据服务中心（deviceStore）开始建立 WebSocket 连接。

2. 界面映射

- 🟢 **已连接 /** 🔴 **未连接**：对应硬件的 **LINK 状态灯**。
- **左侧菜单**：对应控制面板上的 **功能切换按键**。
- **中间内容区**：对应 **功能插卡槽**（插到哪，显哪）。

🦴 静态骨架 (Code Architecture)

关键状态量 (States)

- `currentTime`：[ref] 实时系统时间寄存器。
- `connectionStatus`：[computed] 通信链路状态指示位，派生自 `deviceStore.wsConnected`。
- `timer`：本地计时器句柄，用于管理刷新任务。

核心钩子 (Hooks)

- `onMounted`：**系统上电回调。**
  - 立即执行一次 `updateTime()`。
  - 启动 1000ms 定时中断任务。
  - 指令下发：`deviceStore.connect()`。
- `onUnmounted`：**系统卸载清理。**
  - 停止定时器，防止"死循环"占用资源。

⚙️ 动态运转 (Dynamic Logic)

逻辑流转图 (Data Flow)

1. **上电阶段**：浏览器解析 JS -> 执行 `onMounted` -> 设置 `setInterval` -> 向后端请求连接。
2. **运行阶段**：

- **自增任务**：每秒更新 `currentTime`，由于 Vue 的响应性，UI 自动刷新。
- **状态监听**：`deviceStore` 中的 WS 状态一旦切为连接，Header 上的 `el-tag` 自动变绿。

3. **切换阶段**：用户点击菜单 -> Router 切换中间的 View 卡片 -> `App.vue` 保持不动（保证了通信不断连）。

🎯 解析主线 (Execution Path)

1. Happy Path: 成功的初始化

```
onMounted(() => {
  updateTime();                    // 立即刷新时间，防止初显延迟
  timer = setInterval(updateTime, 1000); // 启动 1Hz 定时任务
  deviceStore.connect();           // 建立 WS 连接，开始接收遥测数据
});
```

2. 关键语法点拨 (Crossover Analogy)

> 💡 *`<router-view />` **C 类比：动态函数指针 / 模块挂载点。** 类似于主循环里有一个 `while(1) {(*current_page_render)(); }`，当你按键切换功能时，只需要修改 `current_page_render` 的指针指向即可。*

> 💡 *`setInterval` **与** `clearInterval` **C 类比：硬件定时器配置。** 必须养成在销毁组件（关机）时关闭定时器的习惯。否则就像单片机里的定时器一直在刷数据，最终会导致浏览器主线程拥堵。*

📂 交付件说明

- **分析文档**：`App.md` (本文件)
- **可视化图**：`App_diagram.html` (交互式时序图)
- **便携文档**：`App.pdf` (由本文档转换生成)